

Rapport de présentation du projet MTI881

Évaluation d'algorithmes d'apprentissage par renforcement pour un agent autonome de football 3D sur Unity

Table des matières

1. Contexte et motivation	2
2. Objectif du projet et questions de recherche.....	2
3. Formulation RL	3
3.1 Environnement.....	3
3.2 État et observations.....	3
3.3 Espace d'actions	4
3.4 Fonction de récompense	4
4. Méthodologie prévue	4
5. Organisation de l'équipe et Calendrier	5

1. Contexte et motivation

Le projet consiste à contrôler un agent dans un jeu de football en 3D dans lequel les cages sont placées en hauteur. Dans ce type d'environnement dynamique et continu, il est très difficile de programmer manuellement toutes les réactions de l'agent (calculer les trajectoires, gérer l'inertie, etc.).

L'apprentissage par renforcement (RL) est une solution particulièrement adaptée à ce problème. Au lieu de coder des règles strictes, l'agent apprendra par ses essais et erreurs en interagissant avec l'environnement pour maximiser ses récompenses.

L'idée de ce projet est notamment inspirée par la vidéo "*AI Learns to Play Soccer (and breaks physics)*", qui présente un environnement de football en 3D similaire à celui envisagé dans ce projet. La vidéo est disponible sur Youtube : <https://www.youtube.com/watch?v=ta99S6Fh53c>

2. Objectif du projet et questions de recherche

L'objectif principal de ce projet est d'évaluer expérimentalement l'efficacité de plusieurs algorithmes d'apprentissage par renforcement dans un environnement de football 3D. Plutôt que d'imposer un algorithme unique, nous chercherons à comparer leurs performances selon plusieurs critères, notamment la vitesse d'apprentissage, la stabilité des performances et la capacité de généralisation.

La question est donc la suivante :

Comment différents algorithmes de renforcement réagissent-ils dans un environnement de football en 3D ? Quelles sont les différences en termes de vitesse d'apprentissage, de stabilité et de généralisation à d'autres configurations de jeu, différentes de celle de l'apprentissage ?

Afin de structurer cet apprentissage et d'évaluer la robustesse des modèles, nous adopterons une méthodologie en plusieurs phases.

L'entraînement débutera dans un environnement restreint en 1 contre 1, afin de permettre l'acquisition des compétences mécaniques de bases nécessaires au contrôle du joueur, telles que le déplacement, l'orientation du ballon et le tir.

Une fois ces fondamentaux validés, la complexité de l'environnement sera augmentée par l'ajout progressif de coéquipiers et d'adversaires, transformant le problème en un défi d'apprentissage par renforcement multi-agent. Cette seconde phase permettra de mesurer l'émergence potentielle de comportements coopératifs selon l'algorithme employé.

Conformément aux exigences d'évaluation, les performances des agents RL seront rigoureusement comparées à une référence sous la forme d'une heuristique réactive simple, programmée pour orienter systématiquement le joueur vers le ballon puis vers le but adverse. Le critère de succès expérimental reposera notamment sur la capacité des modèles à dépasser de manière robuste les performances de cette heuristique, évaluées par des métriques comme le taux de victoire, la différence moyenne de buts, la moyenne de but marquées et encaissés. Mais aussi la performance de l'agent contre des adversaires jamais rencontrés, la récompense moyenne, le temps d'entraînement, etc ...

Nous envisageons de tester également le jeu auto-compétitif, c'est-à-dire de faire jouer le modèle contre lui-même ou contre un autre algorithme de RL.

3. Formulation RL

La modélisation du problème de décision séquentielle est définie comme suit.

3.1 Environnement

Le jeu sera développé sous Unity, et servira d'environnement d'exécution pour les agents. Une couche de communication sera mise en place entre Unity et Python afin de transmettre les observations et les actions. Le protocole de communication sera choisi en fonction des contraintes de performance et de simplicité d'implémentation. Une interface compatible Gymnasium sera développée côté Python afin de standardiser l'interaction avec l'environnement Unity et de permettre l'utilisation de différents algorithmes RL.

3.2 État et observations

L'agent disposera d'une représentation globale et normalisée (valeurs entre -1 et 1) de son environnement afin d'accélérer la convergence et de stabiliser l'apprentissage du réseau de neurones. Le repère spatial sera égocentré (calculé par rapport à la position de l'agent).

L'objectif étant de garder le même réseau de neurone en passant de la phase 1vs1 à la phase en équipe, la problématique de comment gérer l'ajout de joueur se pose. Voici la solution envisagée pour conserver une taille d'observation constante lorsque le nombre de joueurs présents dans l'environnement évolue. Un nombre maximal de coéquipiers et d'adversaires sera défini. Pour chaque emplacement correspondant à un joueur, une information indiquera si celui-ci est effectivement présent dans l'environnement. Les emplacements correspondant à des joueurs absents pourront

ainsi être ignorés par le modèle. Un joueur absent pourra donc par exemple avoir ses valeurs égales à 0 avec un masque de présence nul.

Les observations incluent :

- La position relative et la vitesse 3D du ballon.
- La vitesse 3D de l'agent.
- La position relative et la vitesse 3D des adversaires et des coéquipiers
- La position relative des buts.
- Le temps de jeu et le score.

3.3 Espace d'actions

L'agent pourra effectuer les actions suivantes :

- Se déplacer vers un point.
- Tirer et passer (dans le cadre du jeu en équipe) en variant l'intensité.
- Sauter.
- Tacler (optionnel).

3.4 Fonction de récompense

Au niveau des récompenses, plusieurs types sont à distinguer :

- **Récompenses d'objectif**
- **Récompenses d'apprentissage** : Points attribués si l'agent réduit sa distance avec le ballon, s'il le touche, ou s'il fait un tir cadré. Dans le cadre d'un match en équipes, des points peuvent aussi être attribués pour des passes, des buts précédés de passes, ou autres actions collaboratives.
- **Récompenses défensives** : Une interception défensive est récompensée lorsqu'un agent touche le ballon alors que celui-ci se dirige vers son propre but et aurait pénétré dans la zone de but en l'absence d'interception
- **Pénalités** : Légère perte de points à chaque frame pour encourager la rapidité, et pénalité s'il sort du terrain.

Les valeurs et coefficients de récompense seront déterminés expérimentalement et feront l'objet d'une étude de sensibilité afin d'évaluer leur influence sur l'apprentissage.

4. Méthodologie prévue

Notre logique d'entraînement se déroule en deux temps : sur Unity, notre modèle joue et enregistre différentes parties qui seront envoyées en batch au backend Python où sont implémentées nos fonctions d'entraînement et notre structure de réseau de

neurones, via la librairie PyTorch. Une fois la rétropropagation effectuée et les poids de notre réseau mis à jour, on renvoie le nouveau réseau à Unity qui le met à jour sur le(s) agent(s) en jeu, et on poursuit l'enregistrement de nouveaux épisodes avec un agent légèrement plus performant à chaque épisode.

Cette logique est répétée jusqu'à convergence du modèle ou interruption de l'entraînement par notre équipe, si le modèle est jugé assez performant. Le suivi de la progression du modèle (récompenses, statistiques de jeu) sera réalisé sur Tensorboard. Nous implémenterons également différentes logiques d'exploration / exploitation, ainsi que plusieurs techniques d'accélération d'entraînement (plusieurs agents jouant simultanément...). Cette logique d'entraînement est inspirée du middleware ML-Agents, connu pour l'entraînement par renforcement d'agents sur Unity.

Une fois qu'un modèle satisfaisant est obtenu, nous pouvons le tester en l'affrontant nous-même, où en le comparant à d'autres algorithmes de RL, ou à notre baseline.

5. Organisation de l'équipe et Calendrier

Notre équipe a décidé de s'organiser à travers un canal Whatsapp pour la communication et un Git pour centraliser notre travail.

Le travail sera intégré de manière continue grâce à un pipeline d'intégration et de déploiement continu (CI/CD) mis en place sur GitHub dès le début du projet. Ce pipeline exécutera automatiquement des vérifications, comme la compilation et des tests à chaque contribution. Cela permettra d'assurer une qualité et une rigueur constantes du code.

La validation se fera au moyen de *pull requests*. Aucune modification ne sera fusionnée directement dans la branche principale. Pour préserver la rapidité de développement, une *pull request* pourra être fusionnée par son auteur sans approbation obligatoire d'un autre membre, à condition que toutes les vérifications automatiques du pipeline CI/CD soient réussies.

Au niveau de la répartition des tâches initiale, nous avons essayé de prévoir la charge de travail des différents points du projet. Nous envisageons de faire deux modèles différents afin de les comparer. À partir de cela, nous avons la répartition initiale suivante :

- Alexis : Conception et implémentation modèle 1
- Antoine : Conception et implémentation modèle 1
- Romann : Conception et implémentation modèle 2

- Louis : Conception et implémentation modèle 2, Environnement Unity, conception de la scène, des éléments interactifs et des mécaniques de simulation

Cette organisation est prévisionnelle et sera amenée à évoluer suivant l'avancement du projet.

L'équipe travaille en itérations d'une semaine. Une réunion hebdomadaire réalisée à la suite du cours en présentiel mercredi matin sert à faire le point sur l'avancement, les blocages et les objectifs de la semaine suivante.

Afin de mieux anticiper les différentes étapes du projet, nous avons mis en place le calendrier prévisionnel suivant :

	Proposition de projet												Présentation	
	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	
1. Cadrage et préparation du projet														
Cadrage du projet et réflexion sur les objectifs et la problématique														
Mise en place de l'environnement Git, organisation de l'équipe et rédaction de la proposition de projet														
2. Mise en place de l'environnement														
Mise en place de l'environnement Unity														
Mise en place de l'interface de communication entre Unity et Python														
Mise en place des fonctions nécessaires à l'entraînement														
3. Développement et entraînement initial														
Premiers entraînement des agents dans l'environnement 1 contre 1														
Amélioration et évaluation des premiers modèles en 1 contre 1														
4. Extension de l'environnement														
Implémentation d'un mode permettant à un utilisateur d'affronter l'agent														
Extension vers un environnement 2 contre 2														
5. Expérimentations et évaluation														
Extension progressive vers d'autres configurations de jeu et entraînement des agents														
Expérimentations, comparaison des méthodes et évaluation des performances														
6. Analyse et communication														
Analyse des résultats, finalisation du rapport et préparation de la présentation														